

LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA SORTING ALGORITHM



Nama : Muhammad Alfarisi

NIM : 2025573010051

Kelas : TI /1C

PROGRAM STUDI TEKNIK INFORMATIKA JURUSAN
INFORMASI DAN KOMPUTER POLITEKNIK NEGERI
LHOKSEUMAWE

2026

DAFTAR ISI

LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA SORTING ALGORITHM.....	1
DAFTAR ISI	2
BAB I PENDAHULUAN.....	3
1. Latar Belakang.....	3
2. Identifikasi Masalah	3
3. Rumusan Masalah.....	3
BAB II LANDASAN TEORI.....	4
1. Pengertian Sorting Algorithm	4
2. Tujuan Utama Sorting Algoritma.....	4
3. Jenis-jenis Algoritma Sorting.....	4
BAB III PROSES PRAKTIKUM	5
3.1. Alat Dan Bahan.....	5
3.2. Proses Praktikum.....	5
BAB IV PENUTUP	19
kesimpulan	19

BAB I

PENDAHULUAN

1. Latar Belakang

Berakal dari kebutuhan manusia untuk mengorganisasi, merapikan, dan menyederhanakan data yang acak agar lebih terstruktur.

2. Identifikasi Masalah

- Tujuan dan Tantangan Utama
- Kompleksitas Waktu (Time Complexity)
- Kompleksitas Ruang (Space Complexity)
- Masalah kestabilan (Stability)
- Jenis-jenis Algoritma Berdasarkan Kasusnya

3. Rumusan Masalah

- Perbandingan Kinerja
- Implementasi Pada Struktur Data
- Optimasi Algoritma
- Efisiensi Memori & Kecepatan

BAB II

LANDASAN TEORI

1. Pengertian Sorting Algorithm

Algoritma Sorting adalah Kumpulan langkah logis sistematis yang digunakan untuk menyusun sekumpulan data (seperti angka atau teks) ke dalam urutan tertentu.

2. Tujuan Utama Sorting Algoritma

- Mempercepat pencarian
- Merapikan Data
- Efisiensi Sistem

3. Jenis-jenis Algoritma Sorting

- Bubble sort
- Selection sort
- Insertion sort
- Merge sort & Quick sort

BAB III

PROSES PRAKTIKUM

3.1. Alat Dan Bahan

- Laptop
- Virtual Studio Code
- Git
- GitHub
- Node JS.

3.2. Proses Praktikum

Langkah 1	Membuat folder dalam repository dan masukan ke repository sebelumnya.
-----------	---

```
Mkdir praktikum-5  
Cd praktikum-5
```

Langkah 2	Inisialisasi npm project
-----------	--------------------------

```
npm init -y
```

Langkah 3	Buat file bernama 01-function-dasar.js
-----------	--

```
Touch 01-memorization.js
```

Langkah 4	Isi file 01-memorization.js dengan code berikut
-----------	---

```
// — Higher-Order Function: memo wrapper universal —  
function memoize(fn) {
```

Sorting Algorithm

```
const cache = new Map();
return function (...args) {
  const key = JSON.stringify(args);
  if (cache.has(key)) return cache.get(key);
  const hasil = fn.apply(this, args);
  cache.set(key, hasil);
  return hasil;
};
}
// — 1. Fibonacci —————
function fibNaif(n) {
  if (n <= 1) return n;
  return fibNaif(n - 1) + fibNaif(n - 2);
}
const fibMemo = memoize(function (n) {
  if (n <= 1) return n;
  return fibMemo(n - 1) + fibMemo(n - 2);
});
function fibIteratif(n) {
  if (n <= 1) return n;
  let a = 0,
      b = 1;
  for (let i = 2; i <= n; i++) [a, b] = [b, a + b];
  return b;
}
console.log("=== Fibonacci Benchmark ===");
let t = Date.now();
fibNaif(38);
console.log("Naif O(2n):", Date.now() - t, "ms");
t = Date.now();
fibMemo(38);
console.log("Memo O(n) :", Date.now() - t, "ms");
t = Date.now();
fibIteratif(38);
console.log("Iteratif O(n):", Date.now() - t, "ms");
console.log("Hasil fib(38):", fibIteratif(38));
// — 2. Coin Change (jumlah minimum koin) —————
// coinChange(jumlah, koin) = min koin untuk membuat total
// = jumlah
```

Sorting Algorithm

```
const coinChangeMemo = memoize(function (jumlah, koin) {
  if (jumlah === 0) return 0;
  if (jumlah < 0) return Infinity;
  let min = Infinity;
  for (const k of koin) {
    const sub = coinChangeMemo(jumlah - k, koin);
    if (sub + 1 < min) min = sub + 1;
  }
  return min;
});
console.log("\n=== Coin Change ===");
const koin = [1, 5, 10, 25];
console.log("Koin tersedia:", koin);
console.log("41 sen = min", coinChangeMemo(41, koin),
"koin"); // 4
console.log("30 sen = min", coinChangeMemo(30, koin),
"koin"); // 2
console.log("11 sen = min", coinChangeMemo(11, koin),
"koin"); // 2
// — 3. Longest Common Subsequence —————
const lcsMemo = memoize(function (
  s1,
  s2,
  i = s1.length - 1,
  j = s2.length - 1,
) {
  if (i < 0 || j < 0) return 0;
  if (s1[i] === s2[j]) return 1 + lcsMemo(s1, s2, i - 1, j
- 1);
  return Math.max(lcsMemo(s1, s2, i - 1, j), lcsMemo(s1,
s2, i, j - 1));
});
console.log("\n=== Longest Common Subsequence ===");
console.log("LCS(ABCBDBAB, BDCAB) =", lcsMemo("ABCBDBAB",
"BDCAB")); // 4 (BCAB)
console.log("LCS(AGGTAB, GXTXAYB) =", lcsMemo("AGGTAB",
"GXTXAYB"));
```

Sorting Algorithm

Langkah 5 Simpan lalu jalankan pada tampilan terminal

Node 01-memorization.js

Hasil Run terminal untuk 01-memorization.js

```
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/prak
• $ node 01-memorization.js
=== Fibonacci Benchmark ===
Naif O(2n): 693 ms
Memo O(n) : 0 ms
Iteratif O(n): 0 ms
Hasil fib(38): 39088169

=== Coin Change ===
Koin tersedia: [ 1, 5, 10, 25 ]
41 sen = min 4 koin
30 sen = min 2 koin
11 sen = min 2 koin

=== Longest Common Subsequence ===
LCS(ABCBAD, BDCAB) = 4
LCS(AGGTAB, GXTXAYB) = 4
```

Langkah 6 Buat file dengan nama 02-merhe-sort.js

Touch 02-merche-sort.js

Langkah 7 Isi file 02-merche-sort.js dengan code berikut

```
// — Fungsi merge: gabung dua array terurut —
function merge(kiri, kanan) {
  const hasil = [];
  let i = 0,
      j = 0;
  // bandingkan elemen satu per satu, ambil yang lebih
  kecil
  while (i < kiri.length && j < kanan.length) {
    if (kiri[i] <= kanan[j]) hasil.push(kiri[i++]);
    else hasil.push(kanan[j++]);
  }
```

Sorting Algorithm

```
}
// tambahkan sisa yang belum habis
return [...hasil, ...kiri.slice(i), ...kanan.slice(j)];
}

// — Fungsi mergeSort rekursif —
function mergeSort(arr) {
  // Base case: array 0 atau 1 elemen sudah 'terurut'
  if (arr.length <= 1) return arr;

  const tengah = Math.floor(arr.length / 2);
  const kiri = mergeSort(arr.slice(0, tengah)); // Conquer
  kiri
  const kanan = mergeSort(arr.slice(tengah)); // Conquer
  kanan

  return merge(kiri, kanan); // Combine
}

// — Versi dengan visualisasi step —
let depth = 0;
function mergeSortViz(arr) {
  const pad = " ".repeat(depth);
  console.log(`${pad}DIVIDE: [${arr}]`);
  if (arr.length <= 1) return arr;
  const mid = Math.floor(arr.length / 2);
  depth++;
  const L = mergeSortViz(arr.slice(0, mid));
  const R = mergeSortViz(arr.slice(mid));
  depth--;
  const merged = merge(L, R);
  console.log(`${pad}MERGE : [${L}] + [${R}] =
[${merged}]`);
  return merged;
}

console.log("=== Visualisasi Merge Sort ===");
mergeSortViz([38, 27, 43, 3, 9]);
```

Sorting Algorithm

```
console.log("\n=== Benchmark Merge Sort ===");
const N = 100_000;
const acak = Array.from({ length: N }, () =>
  Math.floor(Math.random() * N));

let t = Date.now();
const sorted = mergeSort(acak);
console.log(`mergeSort(${N} elemen): ${Date.now() - t}
ms`);
console.log("Terurut?", sorted[0] <= sorted[1] && sorted[N
- 2] <= sorted[N-1]);
```

Langkah 8 Simpan lalu jalankan pada tampilan terminal

Node 02-merge-sort.js

Hasil Run terminal untuk 02-merge-sort.js

```
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/prak
$ node 02-merge-sort.js
MERGE : [27,38] + [3,9,43] = [3,9,27
,38,43]

=== Benchmark Merge Sort ===
mergeSort(100000 elemen): 91 ms
Terurut? true
```

Langkah 9 Membuat file 03-quick-sort.js

Touch 03-quick-sort.js

Langkah 10 Isi file 03-quick-sort.js dengan code berikut

```
// — Quick Sort dengan pivot elemen tengah —
function quickSort(arr) {
```

Sorting Algorithm

```
    if (arr.length <= 1) return arr;

    const pivotIdx = Math.floor(arr.length / 2);
    const pivot = arr[pivotIdx];

    const kiri = arr.filter((x, i) => i !== pivotIdx && x <=
pivot);
    const kanan = arr.filter((x, i) => i !== pivotIdx && x >
pivot);

    return [...quickSort(kiri), pivot, ...quickSort(kanan)];
}

// — Quick Sort In-Place (lebih efisien memori) —
function partisi(arr, lo, hi) {
    const pivot = arr[hi]; // pivot = elemen terakhir
    let i = lo - 1;
    for (let j = lo; j < hi; j++) {
        if (arr[j] <= pivot) {
            i++;
            [arr[i], arr[j]] = [arr[j], arr[i]]; // swap
        }
    }
    [arr[i + 1], arr[hi]] = [arr[hi], arr[i + 1]]; //
tempatkan pivot
    return i + 1;
}

function quickSortInPlace(arr, lo = 0, hi = arr.length -
1) {
    if (lo < hi) {
        const p = partisi(arr, lo, hi);
        quickSortInPlace(arr, lo, p - 1);
        quickSortInPlace(arr, p + 1, hi);
    }
    return arr;
}

// — Benchmark: Merge Sort vs Quick Sort —
```

Sorting Algorithm

```
const N = 100_000;
const data = Array.from({ length: N }, () =>
Math.floor(Math.random() * N));

console.log("=== Quick Sort Demonstrasi ===");
console.log(quickSort([64, 34, 25, 12, 22, 11, 90]));

console.log("\n=== Benchmark ===");
let t = Date.now();
const a1 = [...data];
quickSort(a1);
console.log(`quickSort (${N}): ${Date.now() - t} ms`);

t = Date.now();
const a2 = [...data];
quickSortInPlace(a2);
console.log(`quickSortInPlace(${N}): ${Date.now() - t}
ms`);

// Menggunakan built-in sort JavaScript (Timsort - O(n log
n))
t = Date.now();
const a3 = [...data];
a3.sort((a, b) => a - b);
console.log(`Array.sort (builtin)(${N}): ${Date.now() - t}
ms`);

console.log("\n=== Worst Case Quick Sort ===");
// Array sudah terurut = worst case untuk pivot elemen
terakhir
const terurut = Array.from({ length: 10000 }, (_, i) =>
i);
t = Date.now();
// quickSortInPlace([...terurut]); // HATI-HATI: ini bisa
stack overflow!
console.log("(Dilewati untuk menghindari stack overflow
pada n=10000)");
console.log("Ini menunjukkan pentingnya strategi pemilihan
pivot!");
```

Sorting Algorithm

Langkah 11 Simpan dan jalankan pada terminal

Node 03-quick-sort.js

Hasil Run terminal untuk 03-quick-sort.js

```
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/prak
$ node 03-quick-sort.js
=== Quick Sort Demonstrasi ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== Benchmark ===
quickSort (100000): 143 ms
quickSortInPlace(100000): 32 ms
Array.sort (builtin)(100000): 40 ms

=== Worst Case Quick Sort ===
(Dilewati untuk menghindari stack overflow pada n=10000)
Ini menunjukkan pentingnya strategi pemilihan pivot!
```

Langkah 12 Membuat file sortinnng-n2.js

Touch 04-horting-n2.js

Langkah 13 isi file 04-sorting-n2.js dengan code berikut

```
// — Bubble Sort —
function bubbleSort(arr) {
  const a = [...arr];
  for (let i = 0; i < a.length - 1; i++) {
    let sudahTerurut = true;
    for (let j = 0; j < a.length - i - 1; j++) {
      if (a[j] > a[j + 1]) {
        [a[j], a[j + 1]] = [a[j + 1], a[j]];
        sudahTerurut = false;
      }
    }
  }
}
```

Sorting Algorithm

```
    }
    if (sudahTerurut) break; // optimasi early exit
  }
  return a;
}

// — Selection Sort —————
// Cari minimum, tempatkan di depan, ulangi
function selectionSort(arr) {
  const a = [...arr];
  for (let i = 0; i < a.length - 1; i++) {
    let idxMin = i;
    for (let j = i + 1; j < a.length; j++) if (a[j] <
a[idxMin]) idxMin = j;
    if (idxMin !== i) [a[i], a[idxMin]] = [a[idxMin],
a[i]];
  }
  return a;
}

// — Insertion Sort —————
// Ambil elemen satu per satu, sisipkan ke posisi yang
tepat
function insertionSort(arr) {
  const a = [...arr];
  for (let i = 1; i < a.length; i++) {
    const kunci = a[i];
    let j = i - 1;
    while (j >= 0 && a[j] > kunci) {
      a[j + 1] = a[j]; // geser ke kanan
      j--;
    }
    a[j + 1] = kunci; // sisipkan di posisi yang tepat
  }
  return a;
}

// — Demo & Benchmark —————
const contoh = [64, 34, 25, 12, 22, 11, 90];
```

Sorting Algorithm

```
console.log("Input :", contoh);
console.log("Bubble Sort :", bubbleSort(contoh));
console.log("Selection Sort:", selectionSort(contoh));
console.log("Insertion Sort:", insertionSort(contoh));

console.log("\n=== Benchmark n=5000 ===");
const N = 5000;
const acak = Array.from({ length: N }, () =>
Math.floor(Math.random() * N));
["bubbleSort", "selectionSort",
"insertionSort"].forEach((nama) => {
  const fn = { bubbleSort, selectionSort, insertionSort
}[nama];
  const t = Date.now();
  fn([...acak]);
  console.log(`${nama}: ${Date.now() - t} ms`);
});

// Best case Insertion Sort: array hampir terurut
console.log("\n=== Insertion Sort pada data hampir terurut
(n=5000) ===");
const hampirTerurut
= Array.from({ length: N }, (_, i) => i).map((x, i) => i %
50 === 0 ? x + Math.floor(Math.random() * 10) : x, );
let t = Date.now(); insertionSort(hampirTerurut);
console.log("Insertion Sort:", Date.now() - t,
"ms (jauh lebih cepat!!)");
```

Langkah 14 Simpan dan jalankan codenya pada terminal

Node 04-sorting-n2.js

Sorting Algorithm

Hasil Run terminal untuk 04-array-method.js

```
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/prak
• $ node 04-sorting-n2.js
Input : [
  64, 34, 25, 12,
  22, 11, 90
]
Bubble Sort : [
  11, 12, 22, 25,
  34, 64, 90
]
Selection Sort: [
  11, 12, 22, 25,
  34, 64, 90
]
Insertion Sort: [
  11, 12, 22, 25,
  34, 64, 90
]

=== Benchmark n=5000 ===
bubbleSort: 235 ms
selectionSort: 20 ms
insertionSort: 10 ms

=== Insertion Sort pada data hampir
terurut (n=5000) ===
Insertion Sort: 0 ms (jauh lebih cep
at!!)
```

Langkah 15 Buat file bernama 05-counting-sort.js

Touch 05-counting-sort.js

Langkah 16 Isi file 05-counting-sort.js dengan code berikut

```
function countingSort(arr, maxVal) {
  const count = new Array(maxVal + 1).fill(0);
  const output = new Array(arr.length);

  // Hitung kemunculan setiap nilai
  for (const x of arr) count[x]++;
}
```

Sorting Algorithm

```
// Kumulatif: count[i] = jumlah elemen <= i
for (let i = 1; i <= maxVal; i++) count[i] += count[i - 1];
// Bangun output dari kanan ke kiri (agar stabil)
for (let i = arr.length - 1; i >= 0; i--) {
  output[--count[arr[i]]] = arr[i];
}
return output;
}

const nilai = [4, 2, 2, 8, 3, 3, 1, 7, 3, 5];
console.log("Input :", nilai);
console.log("Sorted :", countingSort(nilai, 9));

// Benchmark: Counting Sort vs Quick Sort untuk data integer kecil
const N = 1_000_000;
const data = Array.from({ length: N }, () => Math.floor(Math.random() * 1000));
console.log("\n=== Benchmark n=1,000,000 ===");
let t = Date.now();
countingSort([...data], 999);
console.log("Counting Sort O(n) :", Date.now() - t, "ms");
t = Date.now();
[...data].sort((a, b) => a - b);
console.log("Array.sort (Timsort) :", Date.now() - t, "ms");
```

Langkah 17 Simpan dan jalankan codenya pada terminal

Node 05-counting-sort.js

Sorting Algorithm

Hasil Run terminal untuk 05-counting-sort.js

```
ASUS@LAPTOP-6BI2HB8D MINGW64 /d/2025573010051_Algoritma-Dan-Struktur-Data/prak
• $ node 05-counting-sort.js
Input : [
  4, 2, 2, 8, 3,
  3, 1, 7, 3, 5
]
Sorted : [
  1, 2, 2, 3, 3,
  3, 4, 5, 7, 8
]

=== Benchmark n=1,000,000 ===
Counting Sort O(n) : 65 ms
Array.sort (Timsort) : 438 ms
```

Langkah 18 Membuat program submit ke GitHub

```
git .. // untuk kembali ke root repositori
git status // untuk mengecek status kode yang ingin di
push
git add . praktikum-9/ //untuk menambahkan file yang akan
di push
git commit -m "praktikum 9 : sorting-algorithm" //untuk
menamai kode yang telah di commit untuk di push
git push origin main // untuk push kode yang telah di buat
ke GitHub
```

BAB IV

PENUTUP

kesimpulan

Proses sistematis untuk mengatur kumpulan data menjadi urutan terstruktur (dari kecil kebesar atau sebaliknya). Pemilihan metode sangat bergantung pada ukuran data, kompleksitas waktu (O), dan penggunaan memori.

- Bubble Sort & Selection Sort: sangat cocok untuk dataset kecil atau pembelajaran dasar karena kodenya yang sederhana. Namun, kinerjanya sangat lambat ($O(n^2)$) untuk data berskala besar
- Merge Sort: Menggunakan teknik Divide and Conquer (bagi dan taklukan). Algoritma ini sangat stabil dan efisien untuk memproses data dalam jumlah masif ($O(n \log n)$), meskipun membutuhkan memori tambahan.
- Quick Sort: Merupakan standar andalan dalam pemrograman untuk pengurutan yang cepat dan fleksibel, terutama karena tidak memerlukan memori tambahan dan bekerja sangat baik untuk data berskala besar.